MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: MOHIT SHARMA

Subject & Code: DMW & 5AD5-16

Branch: CSE 5TH SEM

Batch: AI&DS

Semester: Vth

Session: 2022-23(odd Sem.)

Unit No.: 2

Date of Submission: 21-10-2023

Lect.No.	Topics Covered	Page No.	Total pages
1	General approach to classification	1-7	7
2	Decision tree induction	8-10	3
3	Bayes classification methods	11-13	3
4	Advanced classification methods	14-18	5
5	Bayesian belief networks	19-21	3
6	Classification by Backpropagation	22-24	4
7	Support Vector Machines	25-28	4
8	Lazy learners	29-31	3

References Book:

3. **Jiawei Han and Micheline Kamber, Data Mining: Concepts and Techniques, Morgan Kaufmann Publishers, third edition ,2013**
4. **Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar, Introduction to Data Mining, second edition, Pearson, 2019**
3. **Ian. H. Witten, Eibe Frank and Mark. A. Hall, Data Mining: Practical Machine Learning Tools and Techniques, third edition , 2017**

FACULTY **HOD** **DEAN ACADEMICS**

UNIT 2

LECTURE 1

Basic Concept of Classification (Data Mining)

Data Mining: Data mining in general terms means mining or digging deep into data that is in different forms to gain patterns, and to gain knowledge on that pattern. In the process of data mining, large data sets are first sorted, then patterns are identified and relationships are established to perform data analysis and solve problems.

Classification is a task in data mining that involves assigning a class label to each instance in a dataset based on its features. The goal of classification is to build a model that accurately predicts the class labels of new instances based on their features.

There are two main types of classification: binary classification and multi-class classification. Binary classification involves classifying instances into two classes, such as "spam" or "not spam", while multi-class classification involves classifying instances into more than two classes.

The process of building a classification model typically involves the following steps:

DataCollection:
The first step in building a classification model is data collection. In this step, the data relevant to the problem at hand is collected. The data should be representative of the problem and should contain all the necessary attributes and labels needed for classification. The data can be collected from various sources, such as surveys, questionnaires, websites, and databases.

DataPreprocessing:
The second step in building a classification model is data preprocessing. The collected data needs to be preprocessed to ensure its quality. This involves handling missing values, dealing with outliers, and transforming the data into a format suitable for analysis. Data preprocessing also involves converting the data into numerical form, as most classification algorithms require numerical input.

Handling Missing Values: Missing values in the dataset can be handled by replacing them with the mean, median, or mode of the corresponding feature or by removing the entire record.

Dealing with Outliers: Outliers in the dataset can be detected using various statistical techniques such as z-score analysis, boxplots, and scatterplots. Outliers can be removed from the dataset or replaced with the mean, median, or mode of the corresponding feature.

Data Transformation: Data transformation involves scaling or normalizing the data to bring it into a common scale. This is done to ensure that all features have the same level of importance in the analysis.

FeatureSelection:

The third step in building a classification model is feature selection. Feature selection involves identifying the most relevant attributes in the dataset for classification. This can be done using various techniques, such as correlation analysis, information gain, and principal component analysis.

Correlation Analysis: Correlation analysis involves identifying the correlation between the features in the dataset. Features that are highly correlated with each other can be removed as they do not provide additional information for classification.

Information Gain: Information gain is a measure of the amount of information that a feature provides for classification. Features with high information gain are selected for classification.

Principal Component Analysis:

Principal Component Analysis (PCA) is a technique used to reduce the dimensionality of the dataset. PCA identifies the most important features in the dataset and removes the redundant ones.

ModelSelection:

The fourth step in building a classification model is model selection. Model selection involves selecting the appropriate classification algorithm for the problem at hand. There are several algorithms available, such as decision trees, support vector machines, and neural networks.

Decision Trees: Decision trees are a simple yet powerful classification algorithm. They divide the dataset into smaller subsets based on the values of the features and construct a tree-like model that can be used for classification.

Support Vector Machines: Support Vector Machines (SVMs) are a popular classification algorithm used for both linear and nonlinear classification problems. SVMs are based on the concept of maximum margin, which involves finding the hyperplane that maximizes the distance between the two classes.

Neural Networks:

Neural Networks are a powerful classification algorithm that can learn complex patterns in the data. They are inspired by the structure of the human brain and consist of multiple layers of interconnected nodes.

ModelTraining:

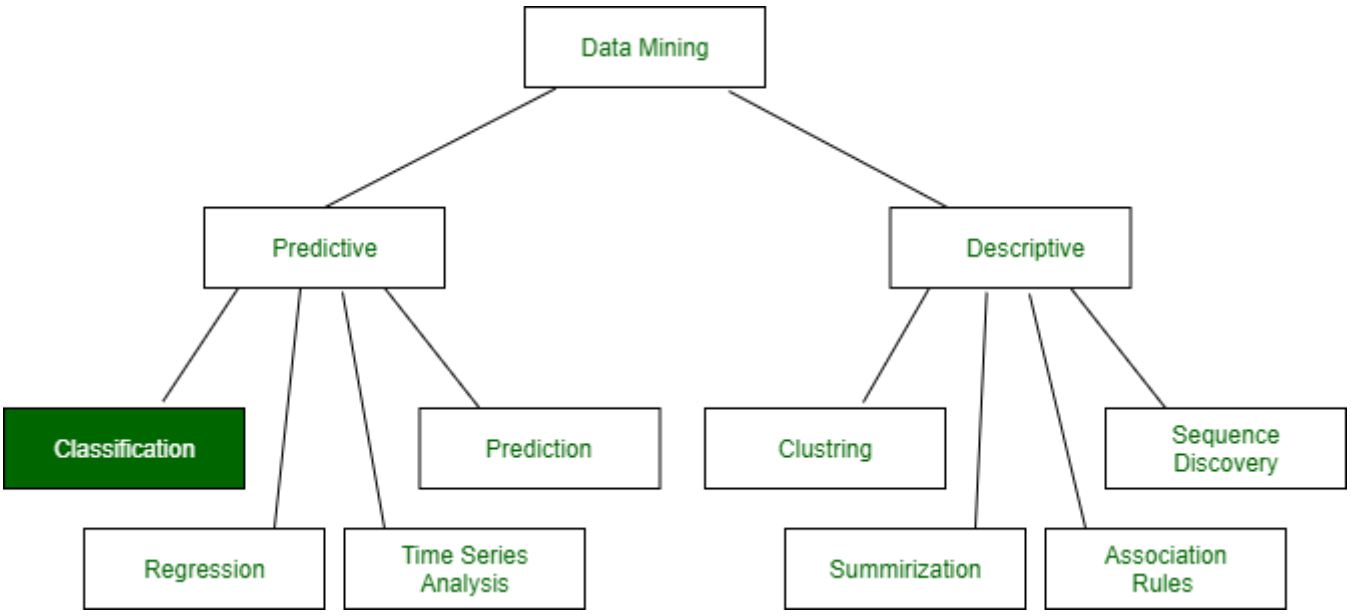
The fifth step in building a classification model is model training. Model training involves using the selected classification algorithm to learn the patterns in the data. The data is divided

into a training set and a validation set. The model is trained using the training set, and its performance is evaluated on the validation set.

ModelEvaluation:

The sixth step in building a classification model is model evaluation. Model evaluation involves assessing the performance of the trained model on a test set. This is done to ensure that the model generalizes well

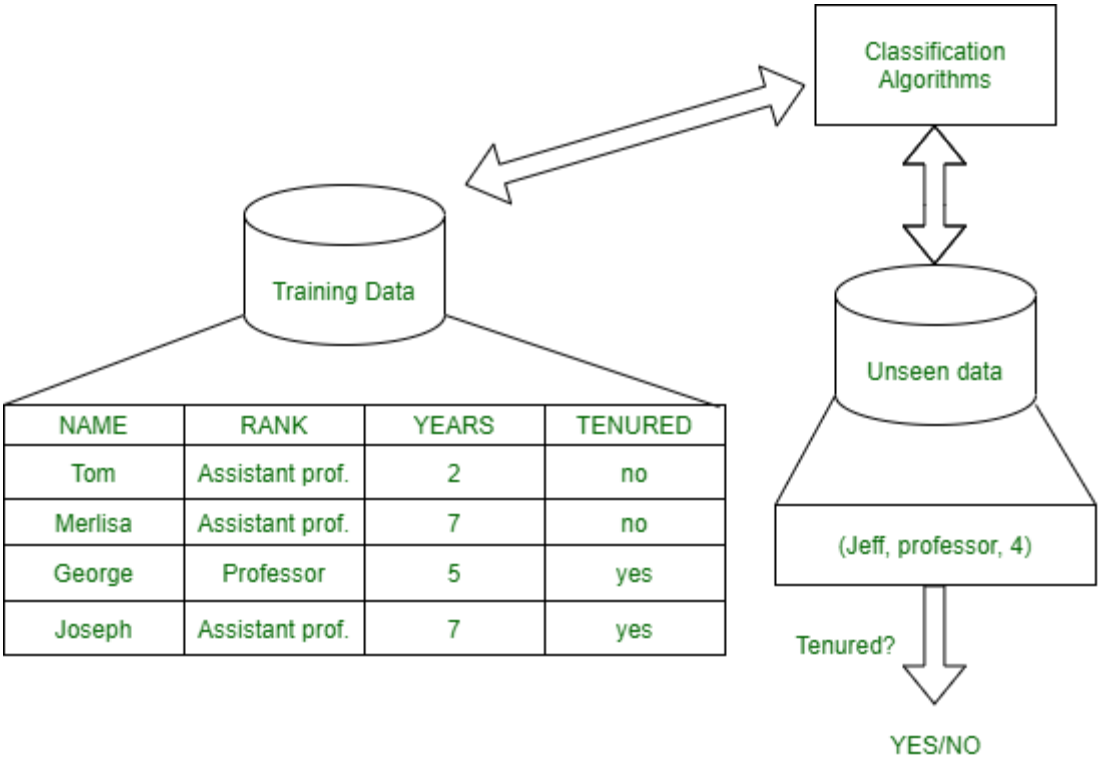
Classification is a widely used technique in data mining and is applied in a variety of domains, such as email filtering, sentiment analysis, and medical diagnosis.



Classification: It is a data analysis task, i.e. the process of finding a model that describes and distinguishes data classes and concepts. Classification is the problem of identifying to which of a set of categories (subpopulations), a new observation belongs to, on the basis of a training set of data containing observations and whose categories membership is known.

Example: Before starting any project, we need to check its feasibility. In this case, a classifier is required to predict class labels such as 'Safe' and 'Risky' for adopting the Project and to further approve it. It is a two-step process such as: / General approach to classification

1. **Learning Step (Training Phase)**: Construction of Classification Model : Different Algorithms are used to build a classifier by making the model learn using the training set available. The model has to be trained for the prediction of accurate results.
2. **Classification Step**: Model used to predict class labels and testing the constructed model on test data and hence estimate the accuracy of the classification rules.



NAME	RANK	YEARS	TENURED
Tom	Assistant prof.	2	no
Merlisa	Assistant prof.	7	no
George	Professor	5	yes
Joseph	Assistant prof.	7	yes

Test data are used to estimate the accuracy of the classification rule

Training and Testing:

Suppose there is a person who is sitting under a fan and the fan starts falling on him, he should get aside in order not to get hurt. So, this is his training part to move away. While Testing if the person sees any heavy object coming towards him or falling on him and moves aside then the system is tested positively and if the person does not move aside then the system is negatively tested.
The same is the case with the data, it should be trained in order to get the accurate and best results.

There are certain data types associated with data mining that actually tells us the format of the file (whether it is in text format or in numerical format).

Attributes – Represents different features of an object. Different types of attributes are:

1. **Binary**: Possesses only two values i.e. True or False
   Example: Suppose there is a survey evaluating some products. We need to check whether it's useful or not. So, the Customer has to answer it in Yes or No.
   Product usefulness: Yes / No
   - **Symmetric**: Both values are equally important in all aspects
   - **Asymmetric**: When both the values may not be important.
2. **Nominal**: When more than two outcomes are possible. It is in Alphabet form rather than being in Integer form.
   Example: One needs to choose some material but of different colors. So, the color might be Yellow, Green, Black, Red.
   Different Colors: Red, Green, Black, Yellow

- **Ordinal**: Values that must have some meaningful order.
  Example: Suppose there are grade sheets of few students which might contain different grades as per their performance such as A, B, C, D
  Grades: A, B, C, D

- **Continuous**: May have an infinite number of values, it is in float type
  Example: Measuring the weight of few Students in a sequence or orderly manner i.e. 50, 51, 52, 53
  Weight: 50, 51, 52, 53

- **Discrete**: Finite number of values.
  Example: Marks of a Student in a few subjects: 65, 70, 75, 80, 90
  Marks: 65, 70, 75, 80, 90

Syntax:

- Mathematical Notation: Classification is based on building a function taking input feature vector "X" and predicting its outcome "Y" (Qualitative response taking values in set C)
- Here Classifier (or model) is used which is a Supervised function, can be designed manually based on the expert's knowledge. It has been constructed to predict class labels (Example: Label – "Yes" or "No" for the approval of some event).

Classifiers can be categorized into two major types:

1. **Discriminative**: It is a very basic classifier and determines just one class for each row of data. It tries to model just by depending on the observed data, depends heavily on the quality of data rather than on distributions.
   Example: Logistic Regression
2. **Generative**: It models the distribution of individual classes and tries to learn the model that generates the data behind the scenes by estimating assumptions and distributions of the model. Used to predict the unseen data.
   Example: Naive Bayes Classifier
   Detecting Spam emails by looking at the previous data. Suppose 100 emails and that too divided in 1:4 i.e. Class A: 25%(Spam emails) and Class B: 75%(Non-Spam emails). Now if a user wants to check that if an email contains the word cheap, then that may be termed as Spam.
   It seems to be that in Class A(i.e. in 25% of data), 20 out of 25 emails are spam and rest not.
   And in Class B(i.e. in 75% of data), 70 out of 75 emails are not spam and rest are spam.
   So, if the email contains the word cheap, what is the probability of it being spam ?? (= 80%)

Classifiers Of Machine Learning:

1. Decision Trees
2. Bayesian Classifiers
3. Neural Networks
4. K-Nearest Neighbour

5. Support Vector Machines
6. Linear Regression
7. Logistic Regression

Associated Tools and Languages: Used to mine/ extract useful information from raw data.

- **Main Languages used**: R, SAS, Python, SQL
- **Major Tools used**: RapidMiner, Orange, KNIME, Spark, Weka
- **Libraries used**: Jupyter, NumPy, Matplotlib, Pandas, ScikitLearn, NLTK, TensorFlow, Seaborn, Basemap, etc.

Real–Life Examples :

- **MarketBasketAnalysis:**
  It is a modeling technique that has been associated with frequent transactions of buying some combination Of items.
  Example: Amazon and many other Retailers use this technique. While viewing some products, certain suggestions for the commodities are shown that some people have bought in the past.
- **WeatherForecasting:**
  Changing Patterns in weather conditions needs to be observed based on parameters such as temperature, humidity, wind direction. This keen observation also requires the use of previous records in order to predict it accurately.

Advantages:

- Mining Based Methods are cost-effective and efficient
- Helps in identifying criminal suspects
- Helps in predicting the risk of diseases
- Helps Banks and Financial Institutions to identify defaulters so that they may approve Cards, Loan, etc.

Disadvantages:
Privacy: When the data is either are chances that a company may give some information about their customers to other vendors or use this information for their profit.
Accuracy Problem: Selection of Accurate model must be there in order to get the best accuracy and result.

APPLICATIONS:

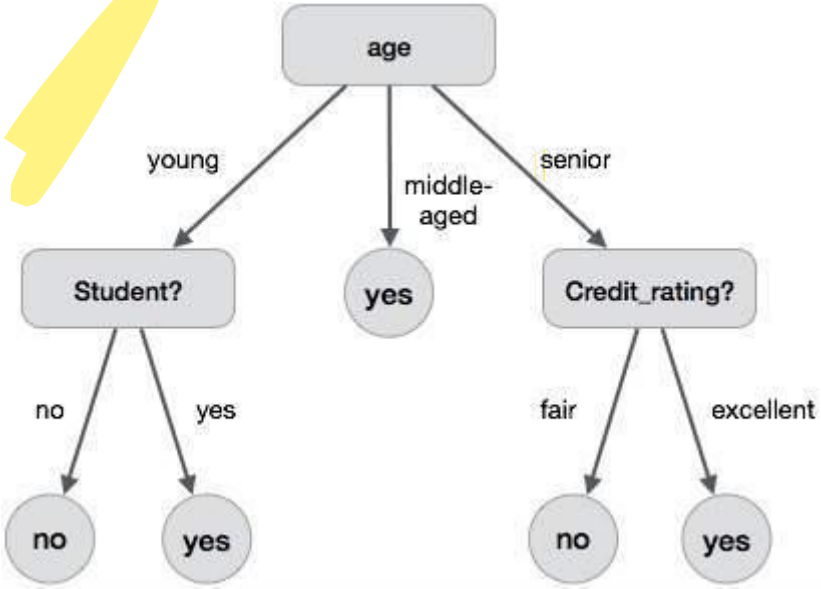
- Marketing and Retailing
- Manufacturing
- Telecommunication Industry
- Intrusion Detection

LECTURE 2

DECISION TREE INDUCTION

A decision tree is a structure that includes a root node, branches, and leaf nodes. Each internal node denotes a test on an attribute, each branch denotes the outcome of a test, and each leaf node holds a class label. The topmost node in the tree is the root node.

The following decision tree is for the concept buy_computer that indicates whether a customer at a company is likely to buy a computer or not. Each internal node represents a test on an attribute. Each leaf node represents a class.



The benefits of having a decision tree are as follows −

- It does not require any domain knowledge.
- It is easy to comprehend.
- The learning and classification steps of a decision tree are simple and fast.

Decision Tree Induction Algorithm

A machine researcher named J. Ross Quinlan in 1980 developed a decision tree algorithm known as ID3 (Iterative Dichotomiser). Later, he presented C4.5, which was the successor of ID3. ID3 and C4.5 adopt a greedy approach. In this algorithm, there is no backtracking; the trees are constructed in a top-down recursive divide-and-conquer manner.

Generating a decision tree form training tuples of data partition D

Algorithm : Generate_decision_tree

Input:

Data partition, D, which is a set of training tuples

and their associated class labels.

attribute_list, the set of candidate attributes.

Attribute selection method, a procedure to determine the

splitting criterion that best partitions that the data

tuples into individual classes. This criterion includes a

splitting_attribute and either a splitting point or splitting subset.

Output:

A Decision Tree

Method

create a node N;

if tuples in D are all of the same class, C then

return N as leaf node labeled with class C;

if attribute_list is empty then

return N as leaf node with labeled

with majority class in D;|| majority voting

apply attribute_selection_method(D, attribute_list)

to find the best splitting_criterion;

label node N with splitting_criterion;

if splitting_attribute is discrete-valued and

multiway splits allowed then // no restricted to binary trees

attribute_list= splitting attribute; // remove splitting attribute

for each outcome j of splitting criterion

// partition the tuples and grow subtrees for each partition

let Dj be the set of data tuples in D satisfying outcome j; // a partition

if Dj is empty then

attach a leaf labeled with the majority

class in D to node N;

else

attach the node returned by Generate

decision tree(Dj, attribute list) to node N;

end for

return N;

Tree Pruning

Tree pruning is performed in order to remove anomalies in the training data due to noise or outliers. The pruned trees are smaller and less complex.

Tree Pruning Approaches

There are two approaches to prune a tree −

- **Pre-pruning** − The tree is pruned by halting its construction early.
- **Post-pruning** - This approach removes a sub-tree from a fully grown tree.

Cost Complexity

The cost complexity is measured by the following two parameters −

- Number of leaves in the tree, and
- Error rate of the tree.

LECTURE 3

BAYESIAN CLASSIFICATION IN DATA MINING

Bayesian classification in data mining is a statistical technique used to classify data based on probabilistic reasoning. It is a type of probabilistic classification that uses Bayes' theorem to predict the probability of a data point belonging to a certain class. The Bayesian classification is a powerful technique for probabilistic inference and decision-making and is widely used in various applications such as medical diagnosis, spam classification, fraud detection, etc.

Introduction to Bayesian Classification in Data Mining

Bayesian classification in data mining is a statistical approach to data classification that uses Bayes' theorem to make predictions about a class of a data point based on observed data. It is a popular data mining and machine learning technique for modelling the probability of certain outcomes and making predictions based on that probability.

The basic idea behind Bayesian classification in data mining is to assign a class label to a new data instance based on the probability that it belongs to a particular class, given the observed data. Bayes' theorem provides a way to compute this probability by multiplying the prior probability of the class (based on previous knowledge or assumptions) by the likelihood of the observed data given that class (conditional probability).

Several types of Bayesian classifiers exist, such as naive Bayes, Bayesian network classifiers, Bayesian logistic regression, etc. Bayesian classification is preferred in many applications because it allows for the incorporation of new data (just by updating the prior probabilities) and can update the probabilities of class labels accordingly.

This is important when new data is constantly being collected, or the underlying distribution may change over time. In contrast, other classification techniques, such as decision trees or support vector machines, do not easily accommodate new data and may require re-training of the entire model to incorporate new information. This can be computationally expensive and time-consuming.

Bayesian classification is a powerful tool for data mining and machine learning and is widely used in many applications, such as spam filtering, text classification, and medical diagnosis. Its ability to incorporate prior knowledge and uncertainty makes it well-suited for real-world problems where data is incomplete or noisy and accurate predictions are critical.

Bayes' Theorem in Data Mining

Bayes' theorem is used in Bayesian classification in data mining, which is a technique for predicting the class label of a new instance based on the probabilities of different class labels and the observed features of the instance. In data mining, Bayes' theorem is used to compute the probability of a hypothesis (such as a class label or a pattern in the data) given some observed event (such as a set of features or attributes). It is named after Reverend Thomas Bayes, an 18th-century British mathematician who first formulated it.

Bayes' theorem states that the probability of a hypothesis H given some observed event E is proportional to the likelihood of the evidence given the hypothesis, multiplied by the prior probability of the hypothesis, as shown below -

$$P(H|E)=P(E)P(E|H)P(H)$$

where $P(H|E)$ is the posterior probability of the hypothesis given the event E, $P(E|H)$ is the likelihood or conditional probability of the event given the hypothesis, $P(H)$ is the prior probability of the hypothesis, and $P(E)$ is the probability of the event.

What is Prior Probability?

Prior probability is a term used in probability theory and statistics that refers to the probability of a hypothesis or event before any event or data is considered. It represents our prior belief or expectation about the likelihood of a hypothesis or event based on previous knowledge or assumptions.

For example, we are interested in the probability of a certain disease in a population. Our prior probability might be based on previous studies or epidemiological data and might be relatively low if the disease is rare. As we collect data from medical tests or patient symptoms, we can update our probability estimate using Bayes' theorem to reflect the new evidence.

What is Posterior Probability?

The posterior probability is a term used in Bayesian inference to refer to the updated probability of a hypothesis, given some observed event or data. It is calculated using Bayes' theorem, which combines the prior probability of the hypothesis with the likelihood of the event to produce an updated or posterior probability.

The posterior probability is important in Bayesian inference because it reflects the latest information about the hypothesis based on the observed data. It can be used to make decisions or predictions and updated further as new data becomes available.

Applications of Bayes' Theorem

Bayes' theorem or Bayesian classification in data mining has a wide range of applications in many fields, including statistics, machine learning, artificial intelligence, natural language processing, medical diagnosis, image and speech recognition, and more. Here are some examples of its applications -

- **Spam filtering** - Bayes' theorem is commonly used in email spam filtering, where it helps to identify emails that are likely to be spam based on the text content and other features.
- **Medical diagnosis** - Bayes' theorem can be used to diagnose medical conditions based on the observed symptoms, test results, and prior knowledge about the prevalence and characteristics of the disease.

- **Risk assessment** - Bayes' theorem can be used to assess the risk of events such as accidents, natural disasters, or financial market fluctuations based on historical data and other relevant factors.
- **Natural language processing** - Bayes' theorem can be used to classify documents, sentiment analysis, and topic modeling in natural language processing applications.
- **Recommendation systems** - Bayes' theorem can be used in recommendation systems like e-commerce websites to suggest products or services to users based on their previous behavior and preferences.
- **Fraud detection** - Bayes' theorem can be used to detect fraudulent behavior, such as credit card or insurance fraud, by analyzing patterns of transactions and other data.

Examples

Problem - Suppose a medical test for a certain disease has a false positive rate of 5% and a false negative rate of 2%. If a person has the disease, there is a 2% chance that the test will come back negative; if a person does not, there is a 5% chance that the test will come back positive. Suppose the disease affects 1% of the population. If a person tests positive for the disease, what is the probability that they have the disease?

Solution - To solve this problem using Bayes' theorem, we can start by defining some events:

- D - the event that a person has the disease
- ~D - the event that a person does not have the disease
- T - the event that a person tests positive for the disease
- ~T - the event that a person tests negative for the disease

We are interested in the probability of event D given the event T, which we can write as $P(D|T)$. Using Bayes' theorem, we can write -

$$P(D|T)=P(T|D)*P(D)/P(T)$$

The first term on the right-hand side of the equation is the probability of a positive test result given that the person has the disease, which we can calculate as -

$$P(T|D)=1-0.02=0.98$$

(2% id FPR, which means that if a person has a disease, then there are 2% chance that the test will come negative)

The second term is the prior probability of the person having the disease, which is given as 1% -

$P(D)=0.01$ (prior probability of disease in given population)

The third term is the probability of a positive test result, which we can calculate using the law of total probability, as shown below -

- $P(T)=P(T|D)*P(D)+P(T|\ D)*P(\ D)$ (It is the sum of the probability of both scenarios when a person tests positive and he may or may not have the disease)
- $P(T)=0.98*0.01+0.05*0.99=0.0593$

Substituting these values into the first equation, we get -

- $P(D|T)=0.98*0.01/0.0593=0.1652$

So the probability that a person has the disease, given that they test positive for it, is approximately 16.52%. This shows that even with a relatively high false positive rate, a positive test result is not a guarantee of having the disease, and further testing or confirmation may be necessary.

LECTURE 4

ADVANCED CLASSIFICATION METHOD

For years, many efforts have been made to develop automated procedures for land use map production using remote sensing image data. However, the situation is still characterised by a considerable operation gap. This gap is even higher since high resolution digital imagery is available. Usual image analysis procedures have had considerable difficulties dealing with the information content of high resolution imagery. Users are moving from pixel oriented classifiers to object oriented analysis systems in order to manage properly the rich information present in those images. Looking for more information is a complementary way of improving the classification results, in parallel to improve image analysis tools.

Analysts looking for land use classification started working with one image using some kind of statistical pixel by pixel classifier with poor or not at all ancillary information. Now the same analysts work with many images, covering the vegetation phenological evolution along the year season, manages different resolution images from different active and passive sensors, works with hierarchical objects having spatial and contextual relationship with their neighbours, combining it with some complementary input as topographic and meteorological data on a Geographic Information System environment.

Nevertheless many questions are still open and we only need to look to the topics concern by the papers presented to this Conference, and also to recent Symposia, or presented recently to Remote Sensing magazines. Without trying to be exhaustive:

o The synergism between classification approaches:

pixel wise classification, context analysis, texture analysis. The right use of segmentation procedures to build objects from pixels components.

o Advanced and practical methodologies of Computer

Assisted Interpretation (CAI) and Analysis of remotely sensed data, and the use of Knowledge

A wise utilisation of information coming from

different sensors. The right classifiers for hyperspectral data. The right classifiers for polarimetric, interferometric and multiband SAR data sets. Multitemporal analysis in order to manage the seasonal evolution of phenomena.

We would like to present next some specific details on each of these topics.

PER-PIXEL VS. OBJECT CLASSIFICATION,

TEXTURE AND CONTEXT ANALYSIS

Very high resolution satellite imagery offers an unseen level of spatial detail which is appropriate for visual interpretation and mapping purposes. On the other hand, difficulties arise when the images has to be classified. The classic per-pixel multispectral classification results in a disgusting salt and pepper effect on complex environment, reducing the land use

maps readability.

Different approaches have been tested but two are basically

followed:

- A segmentation pre-process to build objects and then

classify objects.

- A per-pixel classification and a post-processing land

use parcel building aggregating land cover pixels.

A variety of different classification outputs can be derived from the application of a suite of classifiers to the same data set. The derived classifications may differ greatly in accuracy, on both a per-class and overall basis. By combining the outputs of a set of

classifiers it is possible to derive a classification that is more accurate than any of the individual classifications used. See, for instance: (Briem, 2002), (Ji, 1997), (Liu, 2002), (Steele, 2000). Land cover and land use classification from high spatial

resolution and low spectral resolution images can be done using standard classification techniques. The low spectral resolution can be compensated by the use of texture features, which become meaningful at high spatial resolution. See, for instance: (Zhou, 2003), (Michelet, 2004), (Warner 2005), (Trias-Sanz, 2005).

Image segmentation is usually performed as a pre-processing step for many image understanding applications, for example in some land-cover and land-use classification systems. A segmentation algorithm is used with the expectation that it will divide the image into semantically significant regions, or objects, to be recognized by further processing steps. It is however well known that semantically significant regions are found in an image at different scales of analysis. For a high resolution aerial image, for example, at coarse scales we may find fields, while at finer scales we may find individual trees orplants. Parameters and thresholds in a typical single-scale segmentation algorithm must be tuned to the correct

scale of analysis. However, it is often not possible to determine the correct scale of analysis in advance, because different kinds of images require different scales of analysis, and furthermore in many cases significant objects appear at different scales of analysis in the same image.

In an attempt to overcome this problem, in recent years there has been a trend toward multi-scale or hierarchical segmentation algorithms (Guigues, 2003), (Salembier, 2000). These analyze the image at several different scales at the same time. Their output is not a single partition, but a hierarchy of regions, or some other data structure that captures different partitions for different scales of analysis. As with classical, single-scale, segmentation algorithms, the need arises to evaluate the quality of a multi-scale segmentation against a reference, in order to compare different algorithms, and to select for an algorithm the parameters which are optimal for a given application. Most current segmentation evaluation

methods (Segui, 2003) handle only single scale segmentations, that is, partitions of an image. They usually work by finding correspondences between points in the reference and points in

the edges of the regions given by the segmentation. However, because multi-scale algorithms can deliver arbitrarily fine segmentations the concepts of "correspondence between reference points and segmentation edge points" and of "distance between segmentation edge and reference edge" cannot be easily transposed to the multi-scale case.

COMPUTED ASSISTED INTERPRETATION (CAI)

AND THE USE OF KNOWLEDGE SYSTEMS

An increasing demand for detailed land use maps at regional or national (even continental) level must deal with the huge costs for expert image interpretation, necessary for the extraction of the usually long legend demanded for users. The uses of contextual classifiers on multitemporal images, combined with ancillary information managed in the framework of GIS, have provided some tools to manage the situation. In some European countries environmental agencies ask for country land use maps with a minimum mapping unit of 1 to 5 ha and a legend 50 to 70 items long, covering both land cover and land use classes. There is not an automatic tool to produce this kind of products but there are some advances in order to ga very precise delineation of just a few low level land covers before interpreters should do a more detailed work inside these big parcels.

New classification algorithms like Artificial Immune System (AIS) present innovative approaches to the unsupervised classification of remote sensing images (Zhong 2006).

Spatial data mining, which is also considered as geographical knowledge discovery, is a branch of data mining that has attracted much attention in the recent researches. It puts

emphasis on extraction of interesting and implicit knowledge such as the spatial pattern or other significant mode not explicitly stored in the spatial databases. The main idea of the

research is to utilize spatial data mining techniques to find some interesting knowledge hidden in the spatial data. The extracted knowledge will be use to perform spatial prediction that could make the environmental monitoring task more efficient.

With the rapid development of computer techniques and the data collection and storage techniques, a large amount of spatial data was accumulated. Spatial Data Mining, or knowledge discovery in large spatial databases, is the process of extracting implicit knowledge, spatial relations, or other patterns not explicitly stored in spatial databases. There are many tasks in spatial data mining, such as Spatial Clustering, Spatial Characterization, Spatial Trend Detection, Spatial Classification etc. Also many methods can be used in spatial data mining processes. Decision tree, Bayesian Network, Neural Network, Spatial Analysis and Visualization etc are widely used methods in spatial data mining. They can be combined to complete a special mining task with each other corresponding to the difference of mining targets (Chen, 2005).

There has long been a research goal to produce maps from remotely sensed images in as automated manner as possible. For this goal to be achieved, automated strategies need to be developed that efficiently interpret the information content of highly complex images (Tompkinson, 2005). This investigation takes one approach to implementing the well known principles of top-down and bottom-up reasoning to reliably isolate the geometries of generic objects in the landscape for mapping purposes (Gamba, 2005).

A WISE UTILISATION OF INFORMATION COMING FROM DIFFERENT SENSORS

Almost all classifiers have a relatively good performance with medium resolution multitemporal images (like Landsat TM) and can classify vegetation classes looking for differences on plant phenology. More difficult is to combine SAR and optical images or work with spectral signatures of sensors providing more than 200 bands simultaneously.

The recent developments of the sensor technology resulted in the availability of remote sensing images characterized by very high spectral resolution (hyperspectral images). Nonetheless, the classification of hyperspectral images requires the definition of advanced methodologies capable of dealing with the complex problems induced by the small ratio between the number of training samples and the size of the input feature space. These

problems result in poor estimates of classifier parameters and consequently in low labelling accuracy and unacceptable generalization properties.

LECTURE 5

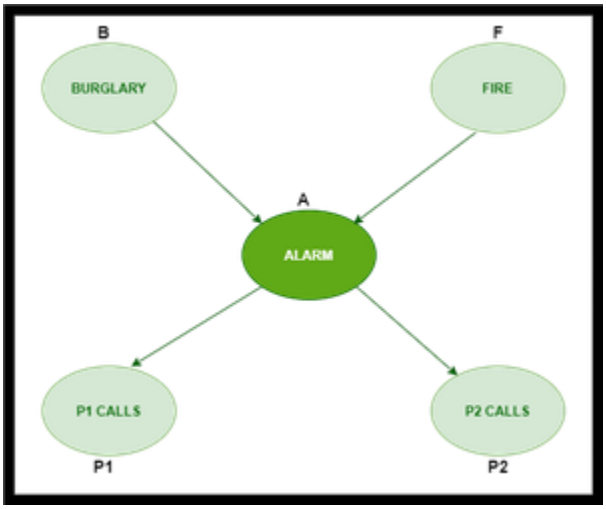
BASIC UNDERSTANDING OF BAYESIAN BELIEF NETWORKS

read from javapoint

Bayesian Belief Network is a graphical representation of different probabilistic relationships among random variables in a particular set. It is a classifier with no dependency on attributes i.e it is condition independent. Due to its feature of joint probability, the probability in Bayesian Belief Network is derived, based on a condition — P(attribute/parent) i.e probability of an attribute, true over parent attribute.

(Note: A classifier assigns data in a collection to desired categories.)

- Consider this example:



- In the above figure, we have an alarm 'A' – a node, say installed in a house of a person 'gfg', which rings upon two probabilities i.e burglary 'B' and fire 'F', which are – parent nodes of the alarm node. The alarm is the parent node of two probabilities P1 calls 'P1' & P2 calls 'P2' person nodes.
- Upon the instance of burglary and fire, 'P1' and 'P2' call person 'gfg', respectively. But, there are few drawbacks in this case, as sometimes 'P1' may forget to call the person 'gfg', even after hearing the alarm, as he has a tendency to forget things, quick. Similarly, 'P2', sometimes fails to call the person 'gfg', as he is only able to hear the alarm, from a certain distance.

Q) Find the probability that 'P1' is true (P1 has called 'gfg'), 'P2' is true (P2 has called 'gfg') when the alarm 'A' rang, but no burglary 'B' and fire 'F' has occurred.

=> **P (P1, P2, A, ~B, ~F)** [where- P1, P2 & A are 'true' events and '~B' & '~F' are 'false' events]

[**Note:** The values mentioned below are neither calculated nor computed. They have observed values]

Burglary 'B' –

- **P (B=T) = 0.001** ('B' is true i.e burglary has occurred)

- **P (B=F) = 0.999** (‘B’ is false i.e burglary has not occurred)

Fire ‘F’ –

- **P (F=T) = 0.002** (‘F’ is true i.e fire has occurred)
- **P (F=F) = <u>0.998</u>** (‘F’ is false i.e fire has not occurred)

Alarm ‘A’ –

B	**F**	**P (A=T)**	**P (A=F)**
T	T	0.95	0.05
T	F	0.94	0.06
F	T	0.29	0.71
F	F	0.001	**<u>0.999</u>**

- The alarm ‘A’ node can be ‘true’ or ‘false’ (i.e may have rung or may not have rung). It has two parent nodes burglary ‘B’ and fire ‘F’ which can be ‘true’ or ‘false’ (i.e may have occurred or may not have occurred) depending upon different conditions.

Person ‘P1’ –

A	**P (P1=T)**	**P (P1=F)**
T	**<u>0.95</u>**	0.05
F	0.05	0.95

- The person ‘P1’ node can be ‘true’ or ‘false’ (i.e may have called the person ‘gfg’ or not) . It has a parent node, the alarm ‘A’, which can be ‘true’ or ‘false’ (i.e may have rung or may not have rung ,upon burglary ‘B’ or fire ‘F’).

Person ‘P2’ –

A	P (P2=T)	P (P2=F)
T	**0.80**	0.20
F	0.01	0.99

- The person ‘P2’ node can be ‘true’ or false’ (i.e may have called the person ‘gfg’ or not). It has a parent node, the alarm ‘A’, which can be ‘true’ or ‘false’ (i.e may have rung or may not have rung, upon burglary ‘B’ or fire ‘F’).

Solution: Considering the observed probabilistic scan –

With respect to the question — **P (P1, P2, A, ~B, ~F)** , we need to get the probability of ‘P1’. We find it with regard to its parent node – alarm ‘A’. To get the probability of ‘P2’, we find it with regard to its parent node — alarm ‘A’.

We find the probability of alarm ‘A’ node with regard to ‘~B’ & ‘~F’ since burglary ‘B’ and fire ‘F’ are parent nodes of alarm ‘A’.

From the observed probabilistic scan, we can deduce –

P (P1, P2, A, ~B, ~F)

= P (P1/A) * P (P2/A) * P (A/~B~F) * P (~B) * P (~F)

= 0.95 * 0.80 * 0.001 * 0.999 * 0.998

= 0.00075

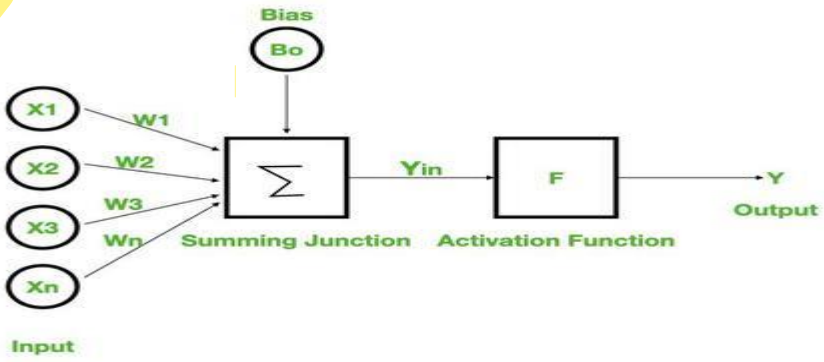
LECTURE 6

BACKPROPAGATION IN DATA MINING

Backpropagation is an algorithm that backpropagates the errors from the output nodes to the input nodes. Therefore, it is simply referred to as the backward propagation of errors. It uses in the vast applications of neural networks in data mining like Character recognition, Signature verification, etc.

Neural Network:

Neural networks are an information processing paradigm inspired by the human nervous system. Just like in the human nervous system, we have biological neurons in the same way in neural networks we have artificial neurons, artificial neurons are mathematical functions derived from biological neurons. The human brain is estimated to have about 10 billion neurons, each connected to an average of 10,000 other neurons. Each neuron receives a signal through a synapse, which controls the effect of the signconcerning on the neuron.



Backpropagation:

Backpropagation is a widely used algorithm for training feedforward neural networks. It computes the gradient of the loss function with respect to the network weights. It is very efficient, rather than naively directly computing the gradient concerning each weight. This efficiency makes it possible to use gradient methods to train multi-layer networks and update weights to minimize loss; variants such as gradient descent or stochastic gradient descent are often used.

The backpropagation algorithm works by computing the gradient of the loss function with respect to each weight via the chain rule, computing the gradient layer by layer, and iterating backward from the last layer to avoid redundant computation of intermediate terms in the chain rule.

Features of Backpropagation:

1. it is the gradient descent method as used in the case of simple perceptron network with the differentiable unit.
2. it is different from other networks in respect to the process by which the weights are calculated during the learning period of the network.
3. training is done in the three stages :
   - the feed-forward of input training pattern
   - the calculation and backpropagation of the error
   - updation of the weight

Working of Backpropagation:

Neural networks use supervised learning to generate output vectors from input vectors that the network operates on. It Compares generated output to the desired output and generates an error report if the result does not match the generated output vector. Then it adjusts the weights according to the bug report to get your desired output.

Backpropagation Algorithm:

Step 1: Inputs X, arrive through the preconnected path.

Step 2: The input is modeled using true weights W. Weights are usually chosen randomly.

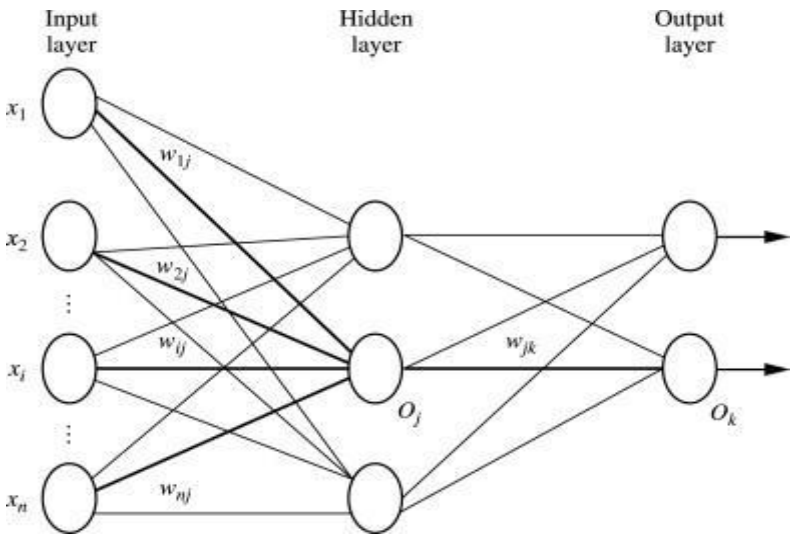
Step 3: Calculate the output of each neuron from the input layer to the hidden layer to the output layer.

Step 4: Calculate the error in the outputs

Backpropagation Error= Actual Output – Desired Output

Step 5: From the output layer, go back to the hidden layer to adjust the weights to reduce the error.

Step 6: Repeat the process until the desired output is achieved.



Parameters :

- x = inputs training vector $x=(x_1,x_2,\ldots\ldots\ldots\ldots x_n)$.
- t = target vector $t=(t_1,t_2\ldots\ldots\ldots\ldots t_n)$.
- δ_k = error at output unit.
- δ_j = error at hidden layer.
- α = learning rate.
- V_{0j} = bias of hidden unit j.

Need for Backpropagation:

Backpropagation is "backpropagation of errors" and is very useful for training neural networks. It's fast, easy to implement, and simple. Backpropagation does not require any parameters to be set, except the number of inputs. Backpropagation is a flexible method because no prior knowledge of the network is required.

Types of Backpropagation

There are two types of backpropagation networks.

- **Static backpropagation:** Static backpropagation is a network designed to map static inputs for static outputs. These types of networks are capable of solving static classification problems such as OCR (Optical Character Recognition).
- **Recurrent backpropagation:** Recursive backpropagation is another network used for fixed-point learning. Activation in recurrent backpropagation is feed-forward until a fixed value is reached. Static backpropagation provides an instant mapping, while recurrent backpropagation does not provide an instant mapping.

Advantages:

- It is simple, fast, and easy to program.
- Only numbers of the input are tuned, not any other parameter.
- It is Flexible and efficient.
- No need for users to learn any special functions.

Disadvantages:

- It is sensitive to noisy data and irregularities. Noisy data can lead to inaccurate results.
- Performance is highly dependent on input data.
- Spending too much time training.
- The matrix-based approach is preferred over a mini-batch.

LECTURE 7

SUPPORT VECTOR MACHINE (SVM) ALGORITHM

-

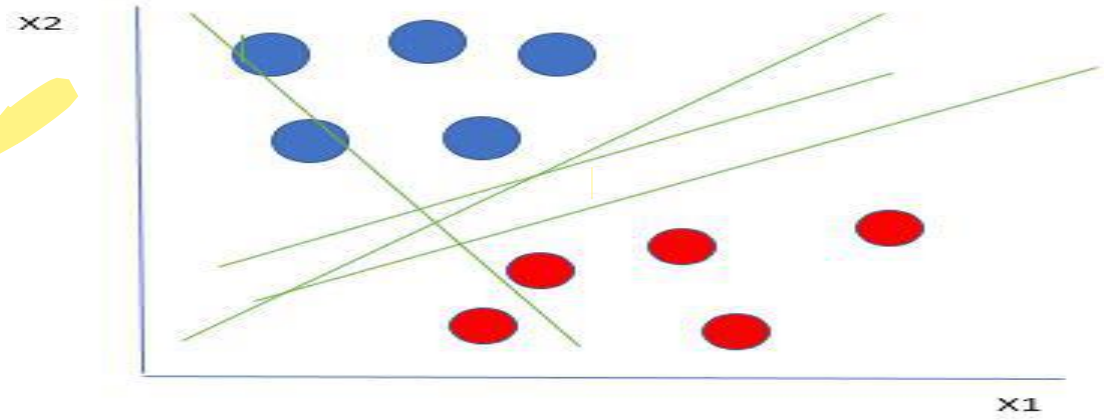
Support Vector Machine (SVM) is a powerful machine learning algorithm used for linear or nonlinear classification, regression, and even outlier detection tasks. SVMs can be used for a variety of tasks, such as text classification, image classification, spam detection, handwriting identification, gene expression analysis, face detection, and anomaly detection. SVMs are adaptable and efficient in a variety of applications because they can manage high-dimensional data and nonlinear relationships.

SVM algorithms are very effective as we try to find the maximum separating hyperplane between the different classes available in the target feature.

Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm used for both classification and regression. Though we say regression problems as well it's best suited for classification. The main objective of the SVM algorithm is to find the optimal hyperplane in an N-dimensional space that can separate the data points in different classes in the feature space. The hyperplane tries that the margin between the closest points of different classes should be as maximum as possible. The dimension of the hyperplane depends upon the number of features. If the number of input features is two, then the hyperplane is just a line. If the number of input features is three, then the hyperplane becomes a 2-D plane. It becomes difficult to imagine when the number of features exceeds three.

Let's consider two independent variables x_1, x_2, and one dependent variable which is either a blue circle or a red circle.

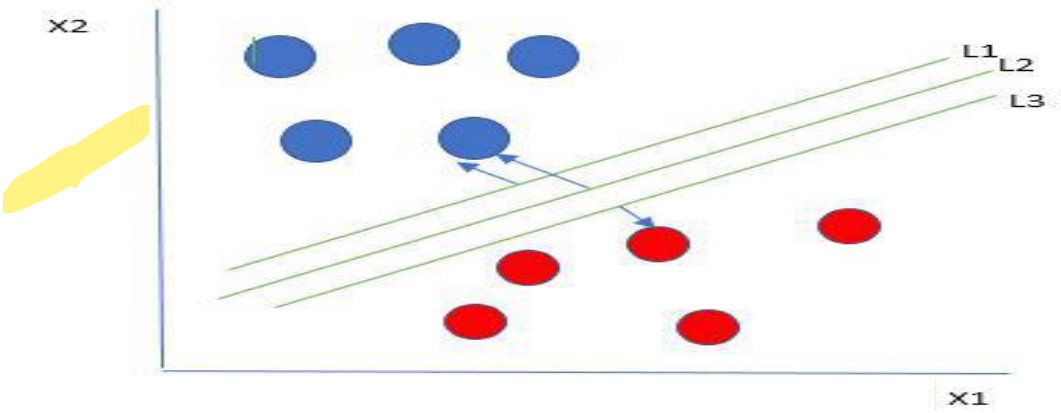


Linearly Separable Data points

From the figure above it's very clear that there are multiple lines (our hyperplane here is a line because we are considering only two input features x_1, x_2) that segregate our data points or do a classification between red and blue circles. So how do we choose the best line or in general the best hyperplane that segregates our data points?

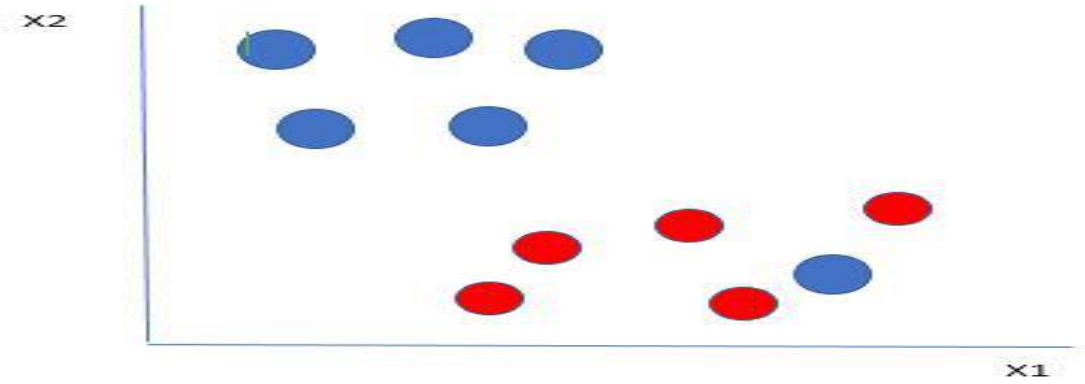
How does SVM work?

One reasonable choice as the best hyperplane is the one that represents the largest separation or margin between the two classes.



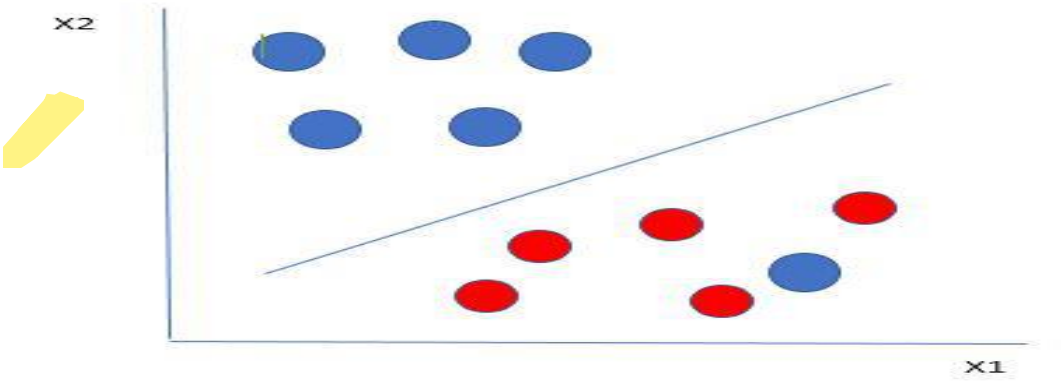
Multiple hyperplanes separate the data from two classes

So we choose the hyperplane whose distance from it to the nearest data point on each side is maximized. If such a hyperplane exists it is known as the **maximum-margin hyperplane/hard margin**. So from the above figure, we choose L2. Let's consider a scenario like shown below



Selecting hyperplane for data with outlier

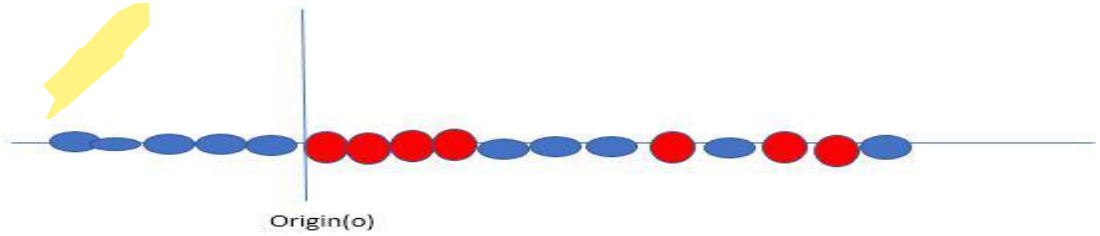
Here we have one blue ball in the boundary of the red ball. So how does SVM classify the data? It's simple! The blue ball in the boundary of red ones is an outlier of blue balls. The SVM algorithm has the characteristics to ignore the outlier and finds the best hyperplane that maximizes the margin. SVM is robust to outliers.



Hyperplane which is the most optimized one

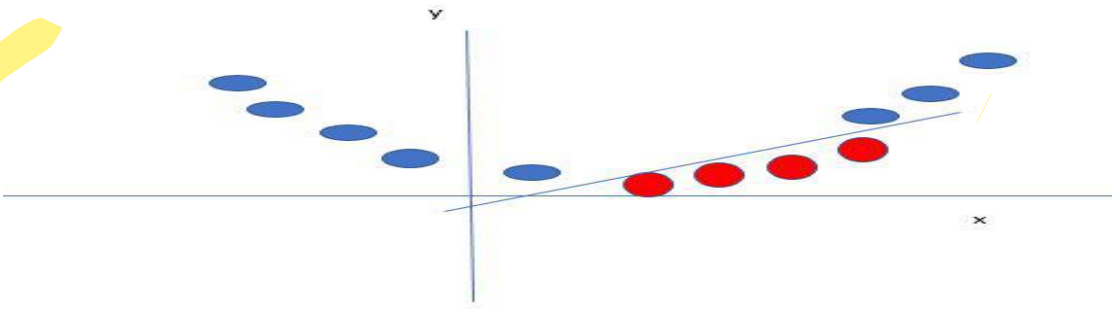
So in this type of data point what SVM does is, finds the maximum margin as done with previous data sets along with that it adds a penalty each time a point crosses the margin. So the margins in these types of cases are called **soft margins**. When there is a soft margin to the data set, the SVM tries to minimize *(1/margin+∧(∑penalty))*. Hinge loss is a commonly used penalty. If no violations no hinge loss.If violations hinge loss proportional to the distance of violation.

Till now, we were talking about linearly separable data(the group of blue balls and red balls are separable by a straight line/linear line). What to do if data are not linearly separable?



Original 1D dataset for classification

Say, our data is shown in the figure above. SVM solves this by creating a new variable using a **kernel**. We call a point x_i on the line and we create a new variable y_i as a function of distance from origin o.so if we plot this we get something like as shown below



Mapping 1D data to 2D to become able to separate the two classes

In this case, the new variable y is created as a function of distance from the origin. A non-linear function that creates a new variable is referred to as a kernel.

Support Vector Machine Terminology

1. **Hyperplane:** Hyperplane is the decision boundary that is used to separate the data points of different classes in a feature space. In the case of linear classifications, it will be a linear equation i.e. wx+b = 0.

2. **Support Vectors:** Support vectors are the closest data points to the hyperplane, which makes a critical role in deciding the hyperplane and margin.

3. **Margin**: Margin is the distance between the support vector and hyperplane. The main objective of the support vector machine algorithm is to maximize the margin. The wider margin indicates better classification performance.

4. **Kernel**: Kernel is the mathematical function, which is used in SVM to map the original input data points into high-dimensional feature spaces, so, that the hyperplane can be easily found out even if the data points are not linearly separable in the original input space. Some of the common kernel functions are linear, polynomial, radial basis function(RBF), and sigmoid.

5. **Hard Margin:** The maximum-margin hyperplane or the hard margin hyperplane is a hyperplane that properly separates the data points of different categories without any misclassifications.

6. **Soft Margin:** When the data is not perfectly separable or contains outliers, SVM permits a soft margin technique. Each data point has a slack variable introduced by the soft-margin SVM formulation, which softens the strict margin requirement and permits certain misclassifications or violations. It discovers a compromise between increasing the margin and reducing violations.

7. **C:** Margin maximisation and misclassification fines are balanced by the regularisation parameter C in SVM. The penalty for going over the margin or misclassifying data items is decided by it. A stricter penalty is imposed with a greater value of C, which results in a smaller margin and perhaps fewer misclassifications.

8. **Hinge Loss:** A typical loss function in SVMs is hinge loss. It punishes incorrect classifications or margin violations. The objective function in SVM is frequently formed by combining it with the regularisation term.

9. **Dual Problem:** A dual Problem of the optimisation problem that requires locating the Lagrange multipliers related to the support vectors can be used to solve SVM. The dual formulation enables the use of kernel tricks and more effective computing.

- **For Soft margin linear SVM classifier:**

- **Dual Problem:** A dual Problem of the optimisation problem that requires locating the Lagrange multipliers related to the support vectors can be used to solve SVM. The optimal Lagrange multipliers $\alpha(i)$ that maximize the following dual objective function

The SVM decision boundary can be described in terms of these optimal Lagrange multipliers and the support vectors once the dual issue has been solved and the optimal Lagrange multipliers have been discovered. The training samples that have $i > 0$ are the support vectors, while the decision boundary is supplied by:

Types of Support Vector Machine

Based on the nature of the decision boundary, Support Vector Machines (SVM) can be divided into two main parts:

- **Linear SVM:** Linear SVMs use a linear decision boundary to separate the data points of different classes. When the data can be precisely linearly separated, linear SVMs are very suitable. This means that a single straight line (in 2D) or a hyperplane (in higher dimensions) can entirely divide the data points into their respective classes. A hyperplane that maximizes the margin between the classes is the decision boundary.

- **Non-Linear SVM:** Non-Linear SVM can be used to classify data when it cannot be separated into two classes by a straight line (in the case of 2D). By using kernel functions, nonlinear SVMs can handle nonlinearly separable data. The original input data is transformed by these kernel functions into a higher-

dimensional feature space, where the data points can be linearly separated. A linear SVM is used to locate a nonlinear decision boundary in this modified space.

Popular kernel functions in SVM

The SVM kernel is a function that takes low-dimensional input space and transforms it into higher-dimensional space, ie it converts nonseparable problems to separable problems. It is mostly useful in non-linear separation problems. Simply put the kernel, does some extremely complex data transformations and then finds out the process to separate the data based on the labels or outputs defined.

Advantages of SVM

- Effective in high-dimensional cases.
- Its memory is efficient as it uses a subset of training points in the decision function called support vectors.
- Different kernel functions can be specified for the decision functions and its possible to specify custom kernels.

LECTURE 8

Lazy learning from screenshot

In machine learning, **lazy learning** is a learning method in which generalization of the training data is, in theory, delayed until a query is made to the system, as opposed to eager learning, where the system tries to generalize the training data before receiving queries.

The primary motivation for employing lazy learning, as in the K-nearest neighbors algorithm, used by online recommendation systems ("people who viewed/purchased/listened to this movie/item/tune also ...") is that the data set is continuously updated with new entries (e.g., new items for sale at Amazon, new movies to view at Netflix, new clips at YouTube, new music at Spotify or Pandora). Because of the continuous update, the "training data" would be rendered obsolete in a relatively short time especially in areas like books and movies, where new best-sellers or hit movies/music are published/released continuously. Therefore, one cannot really talk of a "training phase".

Lazy classifiers are most useful for large, continuously changing datasets with few attributes that are commonly queried. Specifically, even if a large set of attributes exist - for example, books have a year of publication, author/s, publisher, title, edition, ISBN, selling price, etc. - recommendation queries rely on far fewer attributes - e.g., purchase or viewing co-occurrence data, and user ratings of items purchased/viewed.

Advantages

The main advantage gained in employing a lazy learning method is that the target function will be approximated locally, such as in the k-nearest neighbor algorithm. Because the target function is approximated locally for each query to the system, lazy learning systems can simultaneously solve multiple problems and deal successfully with changes in the problem domain. At the same time they can reuse a lot of theoretical and applied results from linear regression modelling (notably PRESS statistic) and control. It is said that the advantage of this system is achieved if the predictions using a single training set are only developed for few objects.This can be demonstrated in the case of the k-NN technique, which is instance-based and function is only estimated locally.

Disadvantages

Theoretical disadvantages with lazy learning include:

- The large space requirement to store the entire training dataset. In practice, this is not an issue because of advances in hardware and the relatively small number of attributes (e.g., as co-occurrence frequency) that need to be stored.
- Particularly noisy training data increases the case base unnecessarily, because no abstraction is made during the training phase. In practice, as stated earlier, lazy learning is applied to situations where any learning performed in advance soon becomes obsolete because of changes in the data. Also, for the problems for which lazy learning is optimal, "noisy" data does not really occur - the purchaser of a book has either bought another book or hasn't.
- Lazy learning methods are usually slower to evaluate. In practice, for very large databases with high concurrency loads, the queries are *not* postponed until actual query time, but recomputed in advance on a periodic basis - e.g., nightly, in anticipation of future queries, and the answers stored. This way, the next time new queries are asked about existing entries in the database, the answers are merely looked up rapidly instead of having to be computed on the fly, which would almost certainly bring a high-concurrency multi-user system to its knees.

- Larger training data also entail increased cost. Particularly, there is the fixed amount of computational cost, where a processor can only process a limited amount of training data points.

There are standard techniques to improve re-computation efficiency so that a particular answer is not recomputed unless the data that impact this answer has changed (e.g., new items, new purchases, new views). In other words, the stored answers are updated incrementally.

This approach, used by large e-commerce or media sites, has long been used in the Entrez portal of the National Center for Biotechnology Information (NCBI) to precompute similarities between the different items in its large datasets: biological sequences, 3-D protein structures, published-article abstracts, etc. Because "find similar" queries are asked so frequently, the NCBI uses highly parallel hardware to perform nightly recomputation. The recomputation is performed only for new entries in the datasets against each other and against existing entries: the similarity between two existing entries need not be recomputed.

Examples of Lazy Learning Methods

- K-nearest neighbors, which is a special case of instance-based learning.
- Local regression.
- Lazy naive Bayes rules, which are extensively used in commercial spam detection software. Here, the spammers keep getting smarter and revising their spamming strategies, and therefore the learning rules must also be continually updated.